

Provenance-Native Applications: A New Paradigm for Multi-Agent Knowledge Work

Sai Eeshwar Divaakar*

Shreyansh Soni

2026

Preprint formatted in the style of *Artificial Intelligence Review* (Springer Nature). Companion to the CLAN format paper.

Abstract

Three decades of knowledge-work software have produced two dominant paradigms: document-centric tools that prioritize portability over intelligence, and platform-centric software-as-a-service that prioritizes intelligence over portability. Neither is equipped for a world in which AI agents are active participants in the authorship of knowledge artifacts, generating structured content, making attributable decisions, and handing off work across organizational roles and pipeline stages. This paper describes a third paradigm: the *provenance-native application*, in which the artifact itself is the primary intelligence container. A self-rendering application template is bundled inside a portable artifact alongside structured AI-readable data and a compressed decision chain; the rendering environment that opens and displays the artifact is thin and stateless and holds no intelligence. We describe the architectural model, the API surface it requires, the multi-stakeholder pipeline it enables, and the range of industry verticals to which it applies. The first reference implementation is built on the CLAN open container format (a companion paper). We argue that the provenance-native paradigm is a strong candidate architectural response to multi-agent knowledge work, and that open format is its natural foundation, while acknowledging the framing is contested.

Keywords: Provenance; Multi-agent systems; Knowledge work; Large language models; Software architecture; Open formats

1 Introduction

Knowledge-work software has evolved in two distinct waves, each defined by where intelligence lives relative to the artifact. The *first wave* of document-centric software produced tools such as word processors, PDF, and EPUB. These tools prioritized portability: a document could travel across organizational boundaries, operating systems, and time. Their failure mode was opacity. A word processor document knows what it contains but not why it contains it, and there is no record of which claims were researched, which were inferred, which were debated, and which were mandated. Intelligence produced the document but did not travel with it. The *second wave* of platform-centric software-as-a-service addressed the intelligence problem by keeping everything in the platform. Collaborative document, design, and customer-relationship platforms built intelligence into the platform layer, including version history, AI suggestions, workflow automation, and role-based access. Their failure mode was portability. Intelligence lives in the platform’s database, so when an artifact leaves the platform by export, download, or handoff to a partner who uses a different tool, it becomes an inert file stripped of its history, rationale, and structure.

The emergence of AI agents as active participants in knowledge work exposes both failure modes at once. An agent that generates a contract clause, a creative brief section, or a financial recommendation produces output that must be attributable (was it the model or the human?),

*Both authors contributed equally. Independent Researcher, Dublin, Ireland.

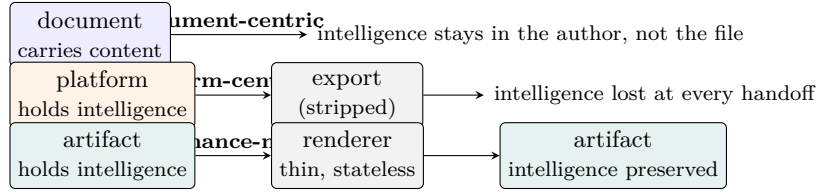


Figure 1: Three waves of knowledge-work software, distinguished by where intelligence lives and what survives a handoff. In the provenance-native model the artifact retains its intelligence across the thin, stateless renderer.

auditable (what rationale produced this conclusion?), portable (the receiving stakeholder may not use the same platform), and resumable (a subsequent agent picking up the work must understand what has already been decided and why). Neither document-centric nor platform-centric tools satisfy all four properties.

This paper describes a third wave, the **provenance-native application**. The central architectural inversion is simple but far-reaching: intelligence belongs to the artifact, not the platform. The rendering environment, that is, whatever software opens the artifact, is a thin, stateless host that executes the template, serves the data, and handles interaction. The artifact itself carries the structured data, the decision history, the agent context, and the lineage graph connecting it to its predecessors and successors in the pipeline. The first reference implementation of this paradigm is built on the CLAN open container format, which we describe in detail in a companion paper. The division of labor between the two papers is deliberate, and we state it to forestall the impression of a single result split in two. The companion paper specifies the format: its container layout, its tamper-evident write semantics, its decision-chain projection, and its conformance requirements. This paper’s contribution is at the paradigm level and is not developed there: the three-wave analysis of where intelligence lives, the template-as-application and commodity-renderer architecture, the `clan://` trust boundary, the inference-proxy pattern, and the cross-vertical multi-stakeholder pipeline. Where the two overlap we describe shared mechanisms only to the depth the argument needs and refer to the companion paper for specification detail. Here we develop the paradigm itself: its principles, its architecture, its multi-stakeholder pipeline, and the range of industry verticals to which it applies. We acknowledge that this framing is contested, since provenance could alternatively live in a protocol layer or a sidecar schema, and we return to that debate in Section 8.

2 The Three Paradigms of Knowledge-Work Applications

Figure 1 summarizes the three waves by where intelligence resides and what survives a handoff. We treat each in turn and then introduce a fourth point of comparison, the agent-framework application, which is AI-native but lacks a portable artifact.

2.1 Document-centric applications

Document-centric tools solved the portability problem definitively. A document created decades ago can still be opened today, the formats are open standards, multiple rendering applications exist, and no vendor platform is required to read the content. Their failure mode for AI-assisted workflows is structural rather than incidental. A document stores content. It does not store the reasoning that produced the content, the agent that generated a specific sentence, the human who overrode an AI suggestion, or the decision that a particular claim is non-negotiable. When a document is handed from one stakeholder to the next, the receiving party sees conclusions without context. For purely human authorship this is often acceptable, because humans can ask the author, but for AI-assisted authorship it becomes critical: a downstream agent resuming work must re-derive all prior context from the content alone, which is expensive, lossy, and unreliable. Document-centric formats are also not structured at the field level. There is no concept of “the `target_audience`

Table 1: The four paradigms against the properties required by AI-assisted, multi-stakeholder knowledge work.

Property	Document	Platform	Agent-framework	Provenance-native
Portable without platform access	✓	×	×	✓
Structured AI-readable data	×	✓ [†]	✓ [†]	✓
Field-level AI attribution	×	×	P [‡]	✓
Human-AI co-authorship model	×	×	×	✓
Decision chain readable by next agent	×	×	×	✓
Self-rendering without platform [§]	✓	×	×	✓
Fork/lineage across pipeline stages	×	✓ [†]	×	✓

✓ satisfied, × not satisfied, P partial. [†] Platform-side only (lost on export). [‡] Present in logs, not in a portable artifact. [§] Interactive self-rendering requires a conformant rendering environment; an external recipient without one relies on a static export (scripts stripped) or manual extraction, so this property is bounded in practice (Sections 3.4 and 9). The robust, unconditional portability is of the data and provenance layers, which are plain text.

field was set by an AI agent on Tuesday based on competitive analysis,” and agents must parse free text to extract structure.

2.2 Platform-centric applications

Platform-centric tools solved the intelligence problem by keeping everything in the platform: version history, model context, structured data, workflow state, and access controls. Intelligence is not lost because it never leaves the platform. The failure mode is portability. When an artifact must cross an organizational boundary, whether from agency to client, from one department to another using different tools, or from a departing employee to a successor, the platform cannot follow, and the exported artifact carries the content and none of the intelligence. For multi-stakeholder AI pipelines this creates a specific breakdown, in that each stage requires access to the same platform to read the decisions of the prior stage. This is often organizationally impossible, and it always creates platform lock-in as a structural dependency. Contemporary AI-writing tools add AI generation on top of this model without changing the underlying architecture, and the exported documents are as inert as they ever were.

2.3 Agent-framework applications

A third category has emerged with multi-agent AI frameworks such as AutoGen [2] and comparable orchestration systems. These applications are genuinely AI-native, with agent chains, tool calls, memory systems, and multi-step reasoning as first-class architectural concerns. Their failure mode is the absence of a portable artifact. The output of an agent-framework run is typically a database record, a log file, or a piece of text, while the reasoning that produced it, including the intermediate agent steps, the tool-call results, and the decision to pursue one approach over another, is stored in the framework’s internal tracing system. That tracing data is not portable, not human-readable without platform access, and not structured for consumption by a subsequent agent operating outside the same framework. The framework optimizes for running the pipeline, not for making the pipeline’s reasoning portable.

Table 1 contrasts the four paradigms against the properties that AI-assisted, multi-stakeholder knowledge work requires.

3 The Provenance-Native Application Model

3.1 Core principles

Four principles define the paradigm.

- P1: Intelligence belongs to the artifact.** The artifact carries its own structured data, decision history, agent context, and lineage. No platform is required to access this intelligence, and a stakeholder who receives the artifact receives the intelligence along with the content.
- P2: Attribution is a tamper-evident precondition of a write.** When an AI agent writes to the artifact’s data layer through the conformant write path, the write is attributed by the format’s protocol rather than by a logging system layered on top, which makes attribution a structural property of the artifact rather than a feature applications may or may not implement. We are precise about the limit, developed in the companion paper: because the artifact is an ordinary archive, the format cannot *prevent* an out-of-band edit, so the guarantee is tamper-evidence (each write records a data digest a validator checks), not tamper-prevention. “Cannot write without attributing” holds for any write through the intended SDK path; a bypass is detectable, not impossible.
- P3: The rendering environment is a commodity.** The application that opens and displays the artifact holds no intelligence. It is a thin, stateless host that reads the artifact’s layers, executes the template, and handles interaction. Any compatible renderer can open any artifact of the format, so the renderer is not the product; the artifact is.
- P4: Human and AI authorship are co-equal, typed, and unified.** A human edit and an AI inference are both entries in the same decision chain with the same attribution structure. The human’s edit is typed as `agent: human, pinned: true`, and the model’s inference is typed as `agent: model-id@version`. Both carry rationale and the specific fields they affected, and both are queryable by any downstream agent.

3.2 The three-layer artifact

The artifact comprises three co-located layers in a single portable container (Figure 2). The presentation layer is a self-contained web application that defines the interaction model for the artifact type, reads data via an injected global, and calls a local API for all writes and AI calls; the AI does not generate this layer. The data layer is structured, schema-versioned content to which AI agents write via RFC 7396 merge patches, with every write atomic and attributed and with human edits tracked as a separate attributed overlay. The intelligence layer holds the append-only, hybrid-compressed decision chain, the manifest (identity, type, lineage), the agent context (working memory for resumption), and structured resumption state. The artifact is self-contained: no network call is needed to render it, no platform account is needed to read its data, and no external service is needed for a downstream agent to understand its decision history.

3.3 Template-as-application

The most important design choice in the model is the co-location of the application template with the data. In content-management systems the template is a platform resource and the data is fetched at render time to populate it, so the template cannot exist without the platform. In the provenance-native model the template lives inside the artifact and defines the full interaction model for its type. A “creative brief” artifact type ships with a template that knows how to render a creative brief, display its structured fields, provide editing surfaces, trigger AI inference, and display the decision chain, whereas a “clinical note” artifact type ships with a completely different template. The templates are different applications, but they share the same container format, write semantics, API surface, and rendering environment. Creating a new artifact is simply copying a template artifact to a new path: there is no setup phase, no platform account, and no provisioning call.

The AI does not generate the template. This is a deliberate constraint, because generating an HTML application from model inference is token-expensive, structurally fragile, and impossible to audit at the field level. The template is authored by developers, versioned, and bundled with the application or fetched from a trusted template registry, and the AI’s job is to write structured data

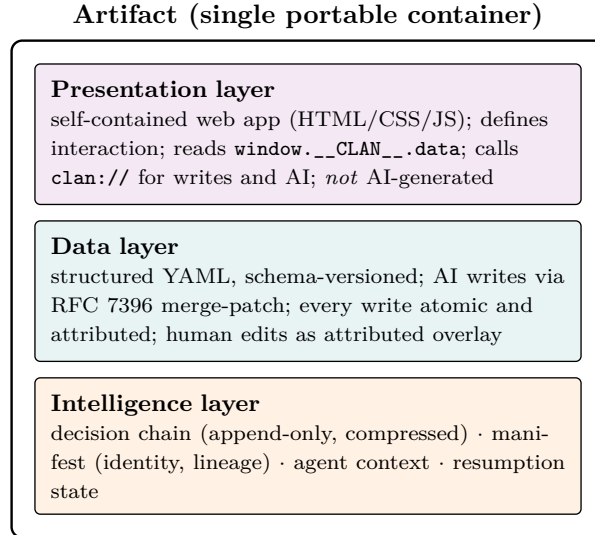


Figure 2: The three-layer provenance-native artifact. The presentation layer renders from the data layer, and the intelligence layer records how the data reached its current state. All three travel together.

into the data layer, not to generate the presentation layer. As in the companion format paper, the template-as-application model carries a known liability, because bundling executable code inside a portable document file has a poor security track record, with PDF JavaScript and document macros as the canonical examples. The implementation mitigates this structurally: templates are authored by known developers, the rendering environment strips `<script>` tags before client delivery, and the sandbox isolates template execution. The more persistent concern is version skew, which is why the rendering layer is best treated as a portability convenience rather than a load-bearing architectural claim. The provenance contributions are independent of whether the presentation layer is present.

3.4 The rendering environment as commodity

The rendering environment is a desktop web-view host (Tauri, Electron, or any WebKit or WebView2 host) that does five things and nothing else: it opens the artifact container and extracts the layers; injects the data layer into the template before render; handles `clan://` custom-scheme requests from the running template (Section 4.1); calls the SDK for data operations; and injects an edit bridge that makes annotated elements editable and routes human edits to the artifact. It holds no intelligence, with no database, no session state, no user-account model, and no model context, so any compatible rendering environment can open any artifact of the format. This has a direct strategic consequence: the rendering environment can be open-sourced and standardized, and multiple environments such as a desktop app, a web viewer, and a mobile reader can serve the same artifacts. Competition is on template quality and AI-pipeline quality, not on format lock-in.

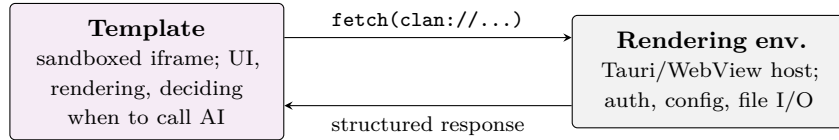
4 Architecture of a Provenance-Native Application

4.1 The artifact API surface

Templates communicate with the rendering environment through a custom URL scheme (`clan://`), which bypasses the `iframe-sandbox` restrictions that would otherwise block local file access while remaining compatible with standard web security models. Templates issue ordinary `fetch()` calls, and the rendering environment intercepts and handles them. Table 2 enumerates the complete contract, and Figure 3 shows the trust boundary it establishes. A template that calls only these endpoints is portable across any compliant rendering environment. The rendering environment is responsible for authentication with external AI services, workspace configuration, and artifact file

Table 2: The `clan://` API surface: the complete contract between a template and any compliant rendering environment.

Template call	Rendering-environment action
POST <code>clan://localhost/patch</code>	Save human edit to <code>patches.yaml</code> ; emit UI update event
POST <code>clan://localhost/patch-data</code>	Call SDK <code>patch_data()</code> ; write to data layer + decision chain
POST <code>clan://localhost/upload-asset</code>	Call SDK <code>patch_asset()</code> ; write to <code>human/assets/</code> ; return internal path
POST <code>clan://localhost/api-proxy</code>	Read workspace config; make authenticated request; keys never leave the host
POST <code>clan://localhost/fork</code>	Call SDK <code>fork()</code> ; create child artifact with embedded parent snapshot
GET <code>clan://localhost/assets/*</code>	Serve a binary asset from inside the artifact ZIP
GET <code>clan://localhost/edit-mode</code>	Return the current edit-mode state



trust boundary: API keys stay host-side

Figure 3: The `clan://` trust boundary. The sandboxed template never holds credentials, and the host mediates every privileged operation.

I/O, while the template is responsible for UI logic, data rendering, user interaction, and deciding when to call AI.

4.2 The AI inference model

API keys for AI services are never stored in the artifact. The artifact stores data and decisions, and keys are a deployment concern. The pattern is a proxy that the rendering environment operates (Figure 4). The template issues an `api-proxy` call with the relevant fields, a prompt-template reference, and any retrieval collection identifiers. The rendering environment reads the workspace configuration for the endpoint, adds the authorization header from its secrets store, and forwards the request to a company proxy server that holds all keys, selects the model, applies rate limits, and logs usage. The structured response returns through the rendering environment to the template, which then writes it via `patch-data` with the model identifier as the attributed agent. Consequently, changing models, rotating keys, or upgrading the AI service requires updating the proxy configuration, not the artifact format, the rendering environment, or any individual artifact. The artifact records what decision was made; it does not record how the inference was provisioned.

4.3 Human–AI co-authorship in the same artifact

Human edits and AI inference both write to the artifact, but through different paths that produce compatible entries in the shared decision chain. An AI inference writes via `patch-data`, producing an entry attributed to the model identifier with its rationale, confidence, and the fields it changed (Listing 1, top). A human edit, routed through the edit bridge into `patches.yaml`, produces an entry attributed to `human` with `pinned: true` (Listing 1, bottom). Both entries share the same structure, and the `pinned: true` flag signals to any downstream agent that the field has been explicitly confirmed by a human and should not be reconsidered. The decision-chain compression strategy, which combines a verbatim window for recent entries, NLP keyword extraction for the older tail, and an exemption for pinned entries, ensures this signal persists at full fidelity across the artifact’s lifetime. As the companion paper details, this compression is applied to the read-time injection an agent consumes, not to the stored chain: the on-disk decision chain retains every full rationale, so the pinned human signal and the complete audit history both persist losslessly

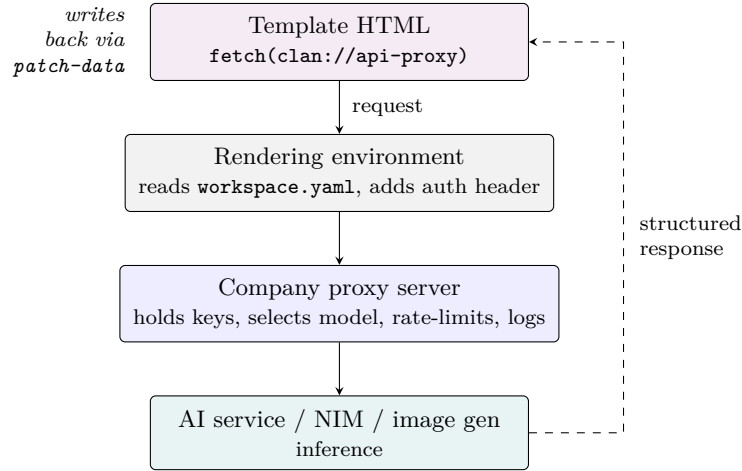


Figure 4: The inference proxy pattern. Credentials and model selection are confined to the proxy server, and the portable artifact records only the resulting attributed decision.

```

- agent_name: "analysis-model@v2.1"
  action: "infer_risk_classification"
  rationale: "Comparable precedents in the reference corpus place this
             submission in Tier 2 on three qualifying criteria. Conf: 0.91."
  fields_changed: ["risk_tier", "applicable_framework", "review_priority"]
  pinned: false
  timestamp: "2026-06-17T14:22:11Z"

- agent_name: "human"
  action: "override"
  rationale: "Confirmed with compliance lead: Tier 1 is mandatory per the 2025
             update. The Tier 2 inference misses the revised threshold."
  fields_changed: ["risk_tier"]
  pinned: true
  timestamp: "2026-06-17T15:03:44Z"

```

Listing 1: Human and AI writes as co-equal, typed entries in one decision chain.

regardless of the projection.

4.4 The workspace and pipeline model

A workspace is a folder containing related artifacts connected by fork lineage (Figure 5). When a downstream role forks a source artifact to begin its work, the fork operation embeds a snapshot of the source data layer at that moment into the child artifact. If the source is subsequently revised, the child is unaffected, because it carries its own copy of the data that was current when its work began. This is the key difference from version-control branching: the fork is asymmetric, so the child knows its parent while the parent does not depend on the child, and the child carries a data snapshot rather than merely a pointer.

5 The Multi-Stakeholder Pipeline: A Walkthrough

To make the architecture concrete, consider a generic three-stage knowledge pipeline applicable to any domain in which structured information passes through multiple specialist roles before a final decision. The pattern of gather evidence, analyse it, then review and decide appears in legal engagements, clinical workflows, financial due diligence, regulatory submissions, and engineering change control. The mechanics below are identical in each case, and only the template design and data schema differ.

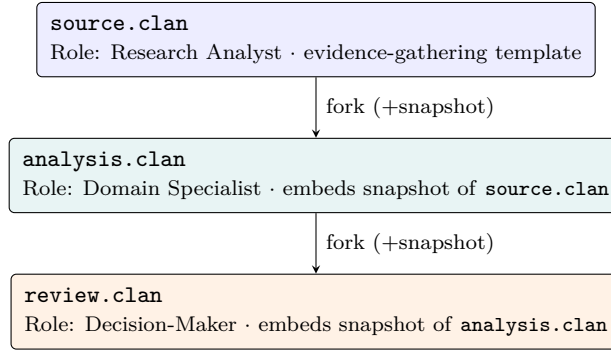


Figure 5: A workspace as a fork-lineage chain. Each child embeds a snapshot of its parent’s data at fork time, so upstream revisions never silently mutate downstream artifacts.

Stage 1: evidence gathering. The Research Analyst opens the rendering environment and creates a source artifact from the evidence-gathering template, which loads a blank schema appropriate to the domain. The analyst enters established facts manually and then triggers AI inference for fields that require synthesis: the template issues an **api-proxy** call with the known fields and a domain-specific prompt template, the model returns structured data, and the template writes the response via **patch-data**. The decision chain records the inference entry (which factors and precedents the model used, with what confidence) alongside the analyst’s manual entries. The analyst then identifies one field that requires a mandatory human override and records it as **agent: human, pinned: true**.

Stage 2: structured analysis. The Domain Specialist opens the source artifact, reads its data and decision chain, and forks it to an analysis artifact, and the fork embeds the source’s current data snapshot. The structured-analysis template shows the source data in a read-only reference panel alongside the specialist’s own writable schema. The specialist triggers AI inference for the assessment summary and risk-factor identification, then reviews the AI output: removing factors that do not apply (recorded as human overrides), adding one the model missed (recorded as **agent: human**), and setting an escalation flag with a rationale. The analysis artifact’s decision chain now contains the AI’s initial reasoning, the field-level human corrections, and the escalation decision.

Stage 3: review and decision. The Decision-Maker forks the analysis artifact to a review artifact. The review template surfaces both the source snapshot (carried forward through the fork chain) and the full analysis data, so the Decision-Maker can read, in a single artifact, the complete chain of reasoning: the original evidence, the analytical interpretation, every AI inference, and every human override across both prior stages, without access to any platform, account, or database. The Decision-Maker approves some fields, overrides others (each with rationale, each pinned), and adds a final disposition.

What a downstream agent sees. When any AI agent opens any artifact in this workspace, it reads the decision chain and has the complete reasoning trail available. Opening the review artifact, an agent sees the source evidence and its initial classification (including the pinned human override that corrected the model), the specialist’s analytical reasoning and corrections, and the decision-maker’s final disposition. Pinned entries appear at full fidelity regardless of when they were made; recent entries are verbatim; older entries are keyword-compressed but semantically complete. The agent does not query a platform database, reconstruct context from version history, or ask a human what was decided and why, because the artifact carries everything required to resume, audit, or explain the work at any point in its lifecycle.

Table 3: Representative verticals on one shared format. Only the template and schema differ across rows.

Vertical	Pipeline	AI role	Why provenance matters
Legal	intake → draft → negotiated → executed	Clause analysis; flag non-standard language; suggest market terms	Records who agreed to what and why at clause level, exactly what a later dispute needs
Medical	intake → diagnosis → treatment → follow-up	Decision support; differential diagnoses; interaction flags (on-prem inference)	Liability and continuity of care require knowing not just the treatment but the reasoning
Finance	research → analysis → recommendation → approved	Synthesize research; valuation estimates; regulatory flags	MiFID II and Reg BI require documenting the basis of recommendations; examination becomes a read
Engineering	requirements → spec → design-review	Consistency checks; contradiction flags; edge-case suggestions	When requirements change, the reason (scope, merge, stakeholder change) is recorded

The pattern generalizes. A legal matter has intake → analysis → opinion; a clinical case has triage → assessment → plan; a financial review has research → analysis → approval; and an engineering change has requirements → impact analysis → change-board approval. In each case the schema, the template, and the AI inference change, while the format, the write semantics, the decision-chain model, the fork/lineage, the rendering environment, and the `clan://` API surface remain identical.

6 Industry Verticals: One Format, Many Applications

The template-as-application model means that different verticals build their own templates and schemas on the same open format, while the rendering environment, the SDK, and the `clan://` API surface remain shared infrastructure. Table 3 maps four representative verticals to their pipelines and the specific reason provenance matters in each.

In legal contract lifecycle management, clauses are elements with stable identifiers, editing a clause produces an attributed patch, and the data schema defines structured fields for each standard clause type. In clinical documentation, the template presents structured SOAP fields, physician edits are element-level patches attributed to the treating physician, and on-premise inference is a hard requirement that the proxy pattern supports by making the endpoint a workspace configuration value. In investment research, the memo template renders data visualizations from the structured layer, and regulatory examination becomes a read operation on the artifact’s intelligence layer rather than a separate audit-trail system. In engineering requirements and design review, a traceability matrix links each requirement to the specification section that addresses it, and a design-review artifact forked from a revised specification carries both the current requirements and the history of how they evolved, including which changes were human decisions (pinned) and which were AI suggestions.

The template ecosystem. Each vertical builds and maintains its own template library. Templates are distributed as template artifacts, that is, CLAN files with `document_type: template`, served by a template registry, and builders can fork and modify templates for their workflow without affecting others. This is the appropriate distribution model for an open format: the format is shared infrastructure, templates are the vertical-specific layer, and template quality is where competitive differentiation happens.

7 Implementation Considerations

Template development. A template is a self-contained web application that calls `clan://` endpoints for all data operations. A developer must know how to read data (access the injected global at load time and re-read on update events), write data via AI (issue a `patch-data` call with RFC 7396 merge-patch semantics, including only changed fields), write data via humans (assign stable identifiers to editable elements, after which the edit bridge handles patch routing), and handle assets (upload and serve via the asset endpoints, storing the internal path in the data layer). The template is authored code, never generated output.

Data-schema design. The schema is the most important design decision, because it is the contract among the template that renders it, the AI that writes to it, and downstream artifacts that inherit a snapshot of it via fork. Schema changes are breaking changes for downstream artifacts that have already forked. The guidelines are to define every field the AI might write as an explicit, typed schema field; to use arrays only where content is genuinely list-valued; to version the schema in the manifest from the first artifact; and to make human-lockable fields identifiable by name rather than position.

Inference configuration and client delivery. The proxy pattern separates the portable artifact from the secret keys. A workspace configuration file stores non-secret configuration, including the endpoint URL, model identifier, retrieval collection identifiers, and prompt-template references, while secrets store the keys. For data residency, the endpoint URL points to an on-premise or regional proxy, and switching from cloud to on-premise inference requires updating one configuration file rather than rebuilding the application or migrating any artifact. When an artifact is ready for external delivery, a stripping operation removes scripts and event handlers from the presentation layer and omits the agent panel, producing a static HTML artifact that any browser can render, and for regulated delivery the structured data layer provides a reliable source for PDF export independent of the rendered HTML.

8 The Strategic Case for Open Format

We flag the nature of this section at the outset: it is an adoption and positioning analysis, not an empirical or formal result. We include it because a format’s viability is inseparable from its adoption dynamics, but a reader seeking only the technical contribution may treat it as context. The argument for open format in this paradigm is strategic rather than altruistic, but the strategy requires honesty about what open format does and does not guarantee. The cautionary example is W3C PROV-O: conceptually well-suited, technically coherent, and near-zero adoption fifteen years on. Openness was not the bottleneck. Vocabulary-only standards without a container format, without tooling, and without a compelling champion failed regardless of technical quality. The instructive counterpoint is RO-Crate [1]: a container-plus-metadata packaging format that did achieve real adoption in research-data communities, which suggests that a concrete container, working tooling, and a host community matter more for adoption than vocabulary elegance. The provenance-native implementation avoids some of these failure modes, since it is a container rather than just a vocabulary, it has a reference implementation, and it is built on ZIP, which every operating system can open. Even so, the adoption challenge is real and should be stated plainly: a format with no readers is a ZIP file.

Open format does not guarantee adoption. What it guarantees is that adoption, if it occurs, cannot be revoked by a single vendor. The specific claim is narrower than “open always wins.” When the format is proprietary, compatibility friction limits the adoption ceiling, and the format holder has a monopoly on compatibility that creates friction at every organizational boundary. When the format is open, compatibility friction is eliminated as a ceiling, and adoption becomes a function of template quality, AI-pipeline quality, and rendering experience, none of which requires lock-in to

compete on. Value therefore accrues to three things: template quality (interaction design, prompt quality, schema completeness, and domain expertise), AI-pipeline quality (the retrieval corpus, prompt templates, and fine-tuned models, with data-flywheel effects), and rendering experience (speed, multi-device support, and integration). None of these competitive dimensions requires format lock-in, and all of them benefit from format ubiquity. An open format with many stakeholders is worth more to any single participant than a proprietary format controlled exclusively.

9 Limitations and Open Problems

Concurrent editing. The model assumes one writer at a time. Two users editing the same artifact simultaneously produces conflicts that the decision-chain model does not resolve. This is acceptable for sequential multi-stakeholder pipelines but fails for real-time collaboration, and addressing it requires CRDTs on the data layer, an operational-transform protocol for patches, or a workspace-level lock. This is the most important open problem in the paradigm.

Rendering-environment availability. An artifact is self-rendering only if the recipient has a compatible environment. The PDF analogy, in which documents are widely readable because readers are ubiquitous, is aspirational rather than current. Achieving comparable ubiquity requires widespread adoption and multiple independent renderer implementations, and until then external stakeholders may need to be supplied with a renderer alongside the artifact.

Large binary asset scale and compliance. The ZIP container is not optimized for multi-gigabyte assets. The STORE method mitigates random-access overhead, but multi-gigabyte artifacts are awkward as portable files, so pipelines with large media may prefer external references with embedded content hashes for integrity. Finally, domain frameworks such as HIPAA, FINRA, and GDPR impose requirements beyond format design, including encryption at rest, access logging, data residency, and right-to-erasure. The format is neutral on these requirements: it does not prevent compliance, but it does not automatically provide it, and each vertical deployment must layer appropriate controls onto the rendering environment and storage infrastructure.

10 Conclusion

The provenance-native application paradigm resolves the tension between portability and intelligence that has characterized knowledge-work software since its inception. By co-locating the application template, the structured data layer, and the decision chain within a single portable artifact, and by reducing the rendering environment to stateless infrastructure, the paradigm enables multi-agent, multi-stakeholder workflows in which intelligence is never stranded at a pipeline boundary. The specific architectural innovations, namely template-as-application, enforced-attribution write semantics, the `clan://` API surface, the proxy pattern for AI keys, and the typed fork/lineage model, together constitute an implementation blueprint that is ready for application across domains. The first reference implementation, built on the CLAN open container format, demonstrates that the model is implementable with current technology. The question for any organization evaluating this paradigm is not whether the problem is real but whether the artifact-centric architectural response is right for its context, and how quickly multi-agent workflows will make intelligence stranding its primary bottleneck. In every domain where AI inference is beginning to participate in knowledge work, that bottleneck is arriving.

Declarations

Funding. No funding was received. **Competing interests.** The authors declare none. **Code availability.** The reference rendering environment, SDK, and CLAN specification are released under MPL-2.0.

References

- [1] Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, Daniel Garijo, Nicola Soranzo, et al. Packaging research artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022.
- [2] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.